University of North Carolina at Charlotte
Restaurant Ordering API

Technical Documentation

ITSC 3155 - Software Engineering

Team Alpha
Date:
Project GitHub: https://github.com/mmallare/ITSC3155_GroupProject

Architecture Overview:

Our online restaurant ordering system API uses a Restful API built with FastAPI and acts as a connection to our MySQL database so users can easily use our system through the SwaggerUI. Our API has five (5) main layers that interact with each other, the models, schemas, routers, controllers, and the database.

A **model** defines how data is stored in a database and includes information such as:

- Table name
- Table columns
- Data types
- Primary and Foreign keys
- Default values
- Relationships with other tables

For example:

```python
class PaymentInformation(Base):  8 usages  Mike
    __tablename__ = "payment_information"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    card_info = Column(String(255), nullable=False)
    status = Column(String(50))
    payment_type = Column(String(50))
    customer_id = Column(Integer, ForeignKey("customers.id"), nullable=False)

    customer = relationship( argument: "Customer", back_populates="payment_information")
```

This is our Payment Information model. It includes the table name and five (5) columns that structure the table in the database. Each column has its own data type (Integer, String, etc). Each model will have a primary key which is, for the most part, the id of that object. One of the columns is customer_id which uses an id from the customer table as a foreign key and this table has a relationship with the customer table so each payment information object will be connected to a customer.

Schemas help define the structure of data entering or leaving the API and in our Schemas are made with Pydantic classes. Pydantic classes are used to create schemas and validate requests and response data.

Schemas main purpose are to validate and format API data and usually include:

- Request-body and Data-type validation
- A required or optional field
- Consistent response formatting
- Separation between the API and database so users aren't directly accessing the database.

For example:

```
class PaymentInformationBase(BaseModel):  2 usages  Mike
    card_info: str
    status: Optional[str] = None
    payment_type: Optional[str] = None
    customer_id: int


class PaymentInformationCreate(PaymentInformationBase):
    pass


class PaymentInformationUpdate(BaseModel):  1 usage  Mike
    card_info: Optional[str] = None
    status: Optional[str] = None
    payment_type: Optional[str] = None
```

These are our Payment Information schemas and they all have different jobs, the base schema creates a structure for other schemas. Notice how the status field is defined with the Optional[str] tag and has no default value, that means that there doesn't need to be a status input for any Payment Information data coming through. Our create schema inherits everything from the base schema and has "pass" which means the class does not add any additional data. Also, the database generates the id for the object so the client does not need to provide one.

Routers define the API endpoints and decide which function handles each HTTP method. Routers main purpose is to receive HTTP requests and route them to the corresponding controller and usually include:

- HTTP methods such as GET, POST, PUT, DELETE
- Endpoint paths
- Path parameters
- Requests and response schemas
- Database session dependencies
- Direction over which controller function should run

For example:

```python
@router.post( path: "/", response_model=schema.PaymentInformation)  Mike
def create(request: schema.PaymentInformationCreate, db: Session = Depends(get_db)):
    return controller.create(db=db, request=request)


@router.get( path: "/", response_model=list[schema.PaymentInformation])  Mike
def read_all(db: Session = Depends(get_db)):
    return controller.read_all(db)


@router.get( path: "/{item_id}", response_model=schema.PaymentInformation)  Mike
def read_one(item_id: int, db: Session = Depends(get_db)):
    return controller.read_one(db, item_id=item_id)


@router.put( path: "/{item_id}", response_model=schema.PaymentInformation)  Mike
def update(item_id: int, request: schema.PaymentInformationUpdate, db: Session = Depends(get_db)):
    return controller.update(db=db, request=request, item_id=item_id)


@router.delete("/{item_id}")  Mike
def delete(item_id: int, db: Session = Depends(get_db)):
    return controller.delete(db=db, item_id=item_id)
```

This is our router for the Payment Information module and each router has its own HTTP method (GET, POST, PUT, DELETE) which listens for a certain request, for example the create router listens for a post request to “/”, and validates the requests function using schema.PaymentInformationCreate (the schema above), obtains the database session, passes the validated data to the Payment Information controller, and formats the returning data using the Payment Information schema as a response model.

Controllers contain the applications CRUD and business logic and perform the requested operations. Controllers act between the router and the database and commonly:

- Create, read, update, and delete records
- Check whether a record in the database exists
- Query the database
- Commit database changes
- Return results or errors

For example:

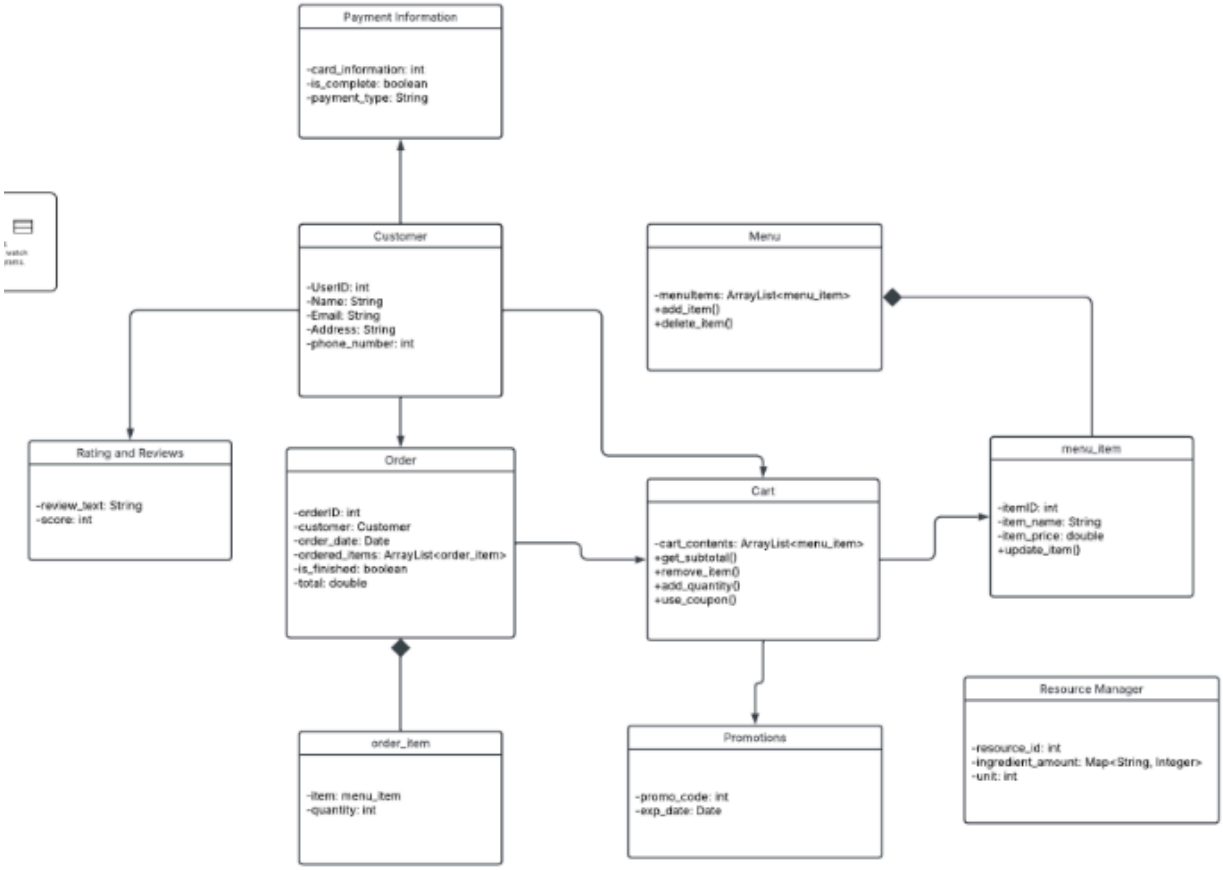
```
45  def update(db: Session, item_id, request):  Mike
46      try:
47          item = db.query(model.PaymentInformation).filter(model.PaymentInformation.id == item_id)
48          if not item.first():
49              raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Id not found!")
50          update_data = request.dict(exclude_unset=True)
51          item.update(update_data, synchronize_session=False)
52          db.commit()
53      except SQLAlchemyError as e:
54          error = str(e.__dict__['orig'])
55          raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=error)
56      return item.first()
```

This is our update controller for the Payment Information module. Line 47 queries the database in search for an object in the Payment Information table that matches id with the id we are searching for which is the item_id parameter and stores that query in “item”. Since we need an existing object to update line 48 and 49 check if our query found a valid record, and if not it raises an HTTPException status code 404, which means the id was not found. If a valid record was found, line 50 turns the user's request into a python dictionary object. Line 51 then uses the update method to update our record and commits the transaction on line 52. If the try block does not go through and there is an error, we move to the except block and a HTTP 400 bad request error is raised.

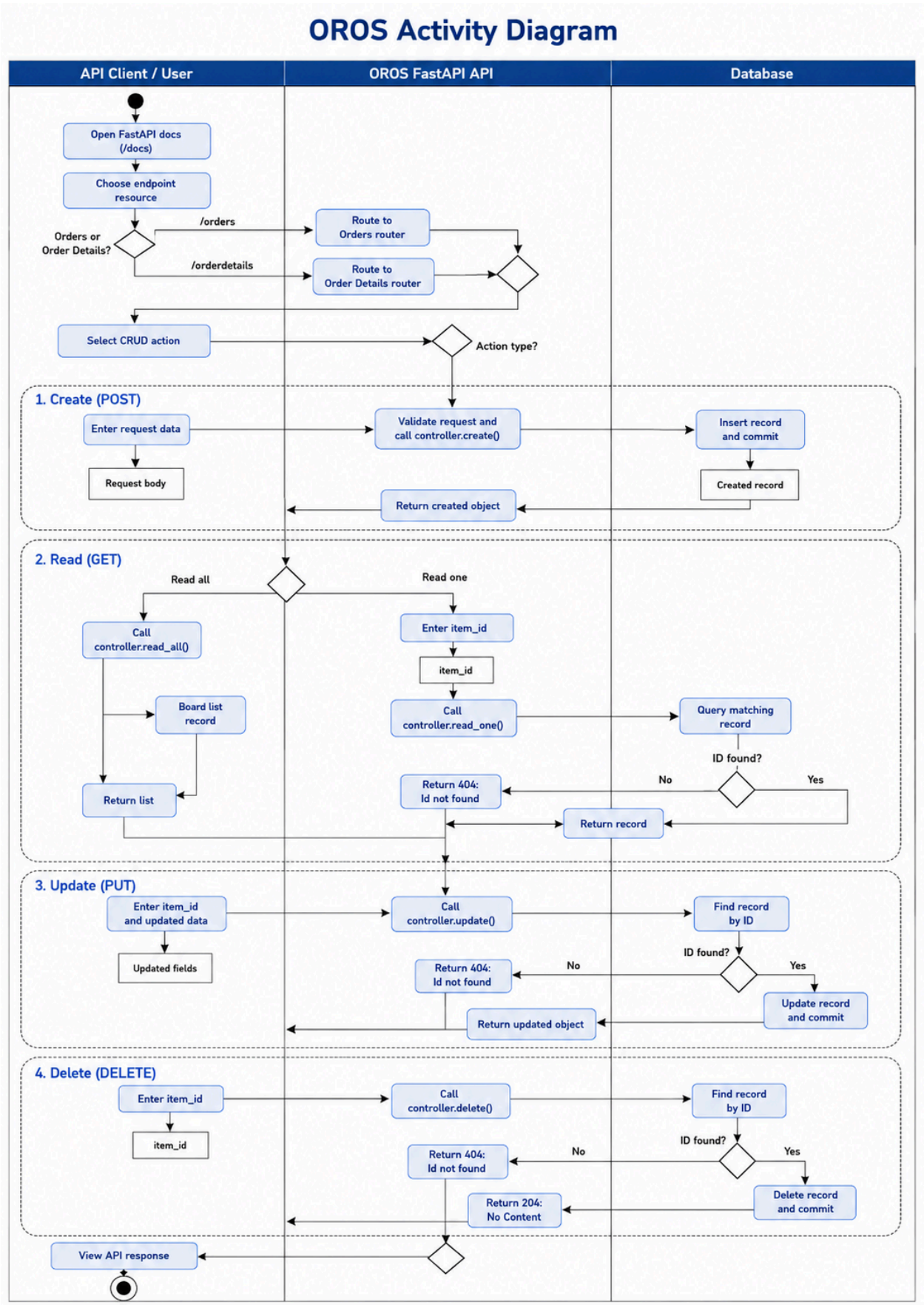
Essentially all of our models, schemas, routers, and controllers are created very similarly to these examples so whenever a user sends a request, the FastAPI router directs it to the correct endpoint, then our Pydantic schema will validate the request data before sending it to the controller. The controller then performs the

requested operation using SQLAlchemy models and a database session before the result is returned to the user as JSON.

Our application has 13 different components and modules that we had to create the models, schemas, routers, and controllers for. Below is a **class diagram** that shows how most of them are connected:



The **activity diagram** below showcases our end-to-end workflow and how CRUD requests flow through our FastAPI application. It follows each request from the API client, through the router and controller, to the database, and how it's returned back to the client as an HTTP response. Included is also requests validation, database operations, commits, and error responses:


OROS Activity Diagram
API Client / User
OROS FastAPI API
Database
Open FastAPI docs (/docs)
Choose endpoint resource
Orders or Order Details?
/orders
/orderdetails
Route to Orders router
Route to Order Details router
Select CRUD action
Action type?
1. Create (POST)
Enter request data
Request body
Validate request and call controller.create()
Insert record and commit
Created record
Return created object
2. Read (GET)
Read all
Read one
Call controller.read_all()
Board list record
Return list
Enter item_id
item_id
Call controller.read_one()
Query matching record
ID found?
No
Yes
Return 404: Id not found
Return record
3. Update (PUT)
Enter item_id and updated data
Updated fields
Call controller.update()
Find record by ID
ID found?
No
Yes
Return 404: Id not found
Update record and commit
Return updated object
4. Delete (DELETE)
Enter item_id
item_id
Call controller.delete()
Find record by ID
ID found?
No
Yes
Return 404: Id not found
Delete record and commit
Return 204: No Content
View API response

Endpoint Documentation:

FastAPI 0.1.0 OAS 3.1

/openapi.json

Orders

- GET /orders/ Read All
- POST /orders/ Create
- GET /orders/track/{tracking_number} Read By Tracking Number
- GET /orders/{item_id} Read One
- PUT /orders/{item_id} Update
- DELETE /orders/{item_id} Delete
- PATCH /orders/{item_id}/status Update Status

Order Details

- GET /orderdetails/ Read All
- POST /orderdetails/ Create
- GET /orderdetails/{item_id} Read One
- PUT /orderdetails/{item_id} Update
- DELETE /orderdetails/{item_id} Delete

Carts

- GET /carts/ Read All
- POST /carts/ Create
- GET /carts/{cart_id} Read One
- PUT /carts/{cart_id} Update
- DELETE /carts/{cart_id} Delete

Payment Information

- GET /payment-information/ Read All
- POST /payment-information/ Create
- GET /payment-information/{item_id} Read One
- PUT /payment-information/{item_id} Update
- DELETE /payment-information/{item_id} Delete

Recipes

- GET /recipes/ Read All
- POST /recipes/ Create
- GET /recipes/{item_id} Read One
- PUT /recipes/{item_id} Update
- DELETE /recipes/{item_id} Delete

Resources

- GET /resources/ Read All
- POST /resources/ Create
- GET /resources/{item_id} Read One
- PUT /resources/{item_id} Update
- DELETE /resources/{item_id} Delete

Sandwiches

- GET /sandwiches/ Read All
- POST /sandwiches/ Create
- GET /sandwiches/{item_id} Read One
- PUT /sandwiches/{item_id} Update

Sandwiches

- GET /sandwiches/ Read All
- POST /sandwiches/ Create
- GET /sandwiches/{item_id} Read One
- PUT /sandwiches/{item_id} Update
- DELETE /sandwiches/{item_id} Delete

Statistics

Menu Items

- GET /menu-items/ Read All
- POST /menu-items/ Create
- GET /menu-items/{item_id} Read One
- PUT /menu-items/{item_id} Update
- DELETE /menu-items/{item_id} Delete

Resource Management

- GET /resource-management/ Read All
- POST /resource-management/ Create
- GET /resource-management/{item_id} Read One
- PUT /resource-management/{item_id} Update
- DELETE /resource-management/{item_id} Delete

Promotions

- GET /promotions/ Read All
- POST /promotions/ Create
- GET /promotions/validate/{promo_code} Validate Code
- GET /promotions/{item_id} Read One
- PUT /promotions/{item_id} Update
- DELETE /promotions/{item_id} Delete

Ratings and Reviews

- GET /ratings-reviews/ Read All
- POST /ratings-reviews/ Create
- GET /ratings-reviews/{item_id} Read One
- PUT /ratings-reviews/{item_id} Update
- DELETE /ratings-reviews/{item_id} Delete

Customers

- GET /customers/ Read All
- POST /customers/ Create
- GET /customers/{item_id} Read One
- PUT /customers/{item_id} Update
- DELETE /customers/{item_id} Delete

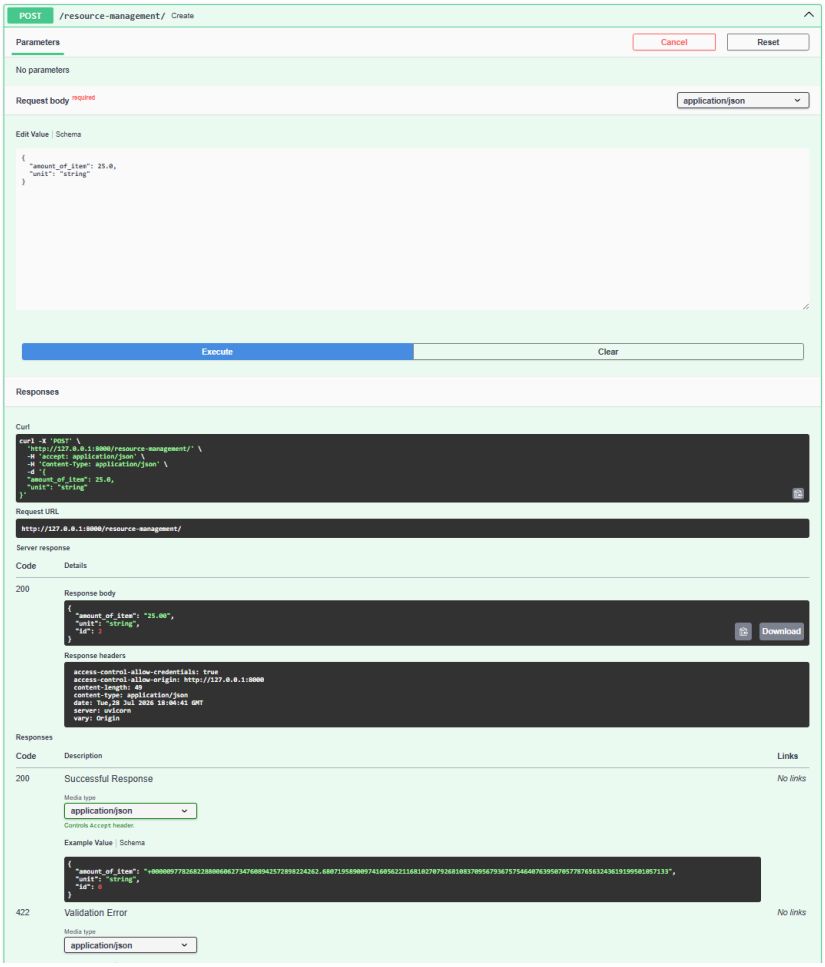
The project registers 13 API router Groups. Each main resource supports CRUD operations through the POST,GET, PUT and DELETE functions. Orders also support tracking and status updates, while promotions support code validation

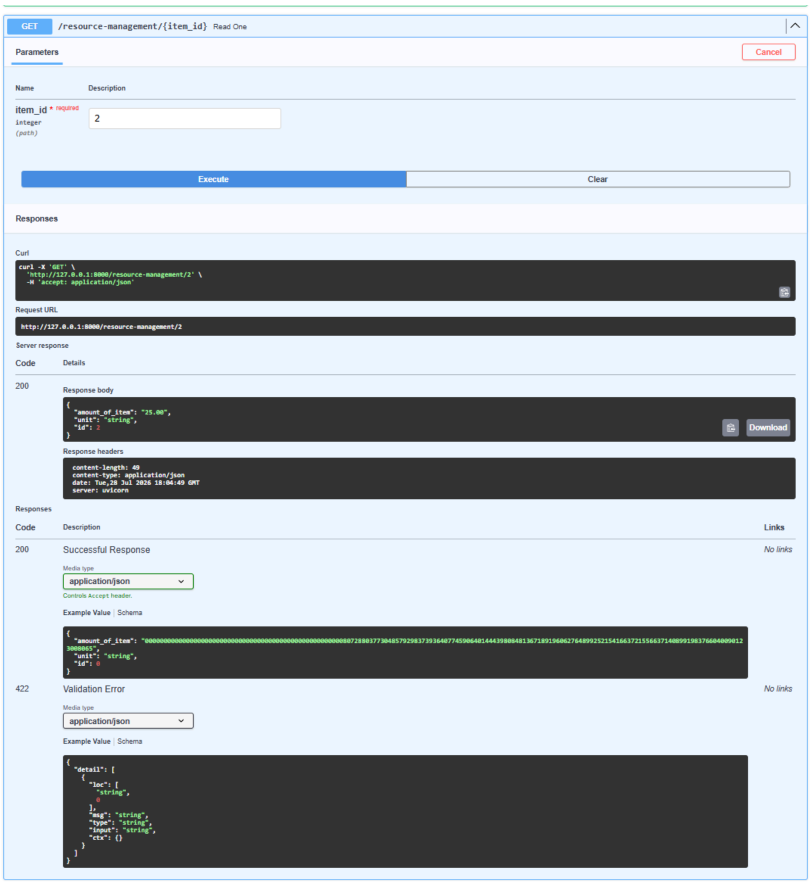
(routers/index.py)

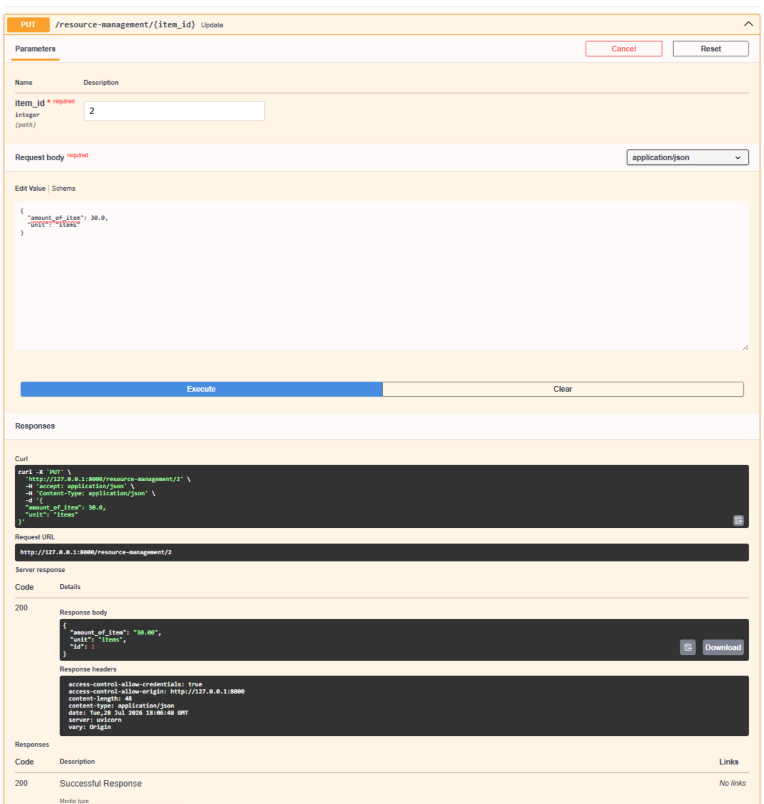
README.md | index.py | config.py | model_loader.py | database.py

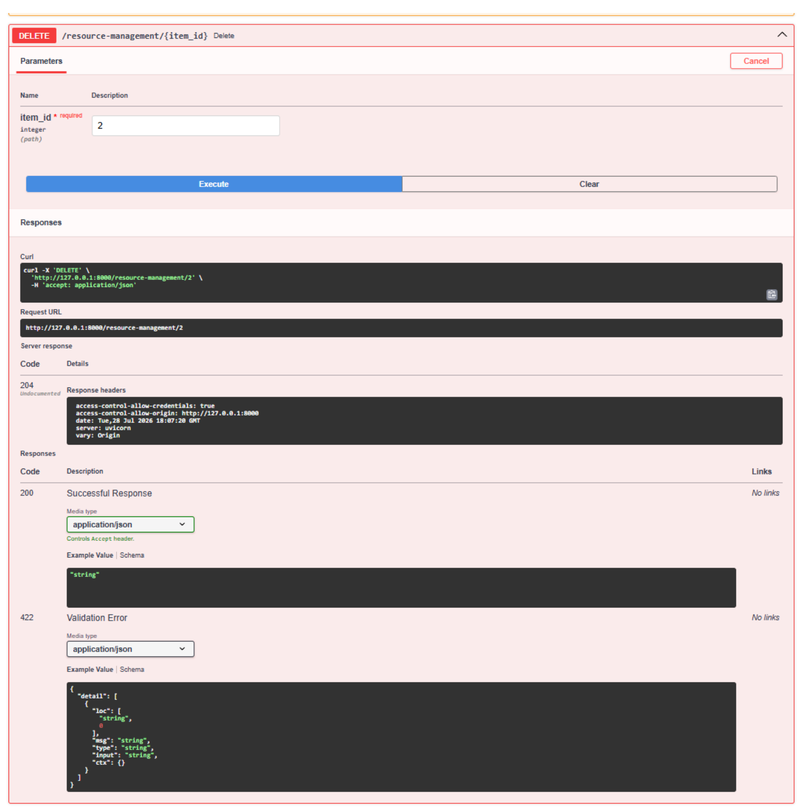
```python
from . import orders, order_details, payment_info, recipes, resources, sandwiches, cart, statistics, menu_items, resource_management, promotions, ratings_reviews, customers


def load_routes(app, include_router): 1 usage  Mike +4
    app.include_router(orders.router)
    app.include_router(order_details.router)
    app.include_router(cart.router)
    app.include_router(payment_info.router)
    app.include_router(recipes.router)
    app.include_router(resources.router)
    app.include_router(sandwiches.router)
    app.include_router(statistics.router)
    app.include_router(menu_items.router)
    app.include_router(resource_management.router)
    app.include_router(promotions.router)
    app.include_router(ratings_reviews.router)
    app.include_router(customers.router)

```

Manual CRUD implementation test performed through FastAPI Swagger UI.

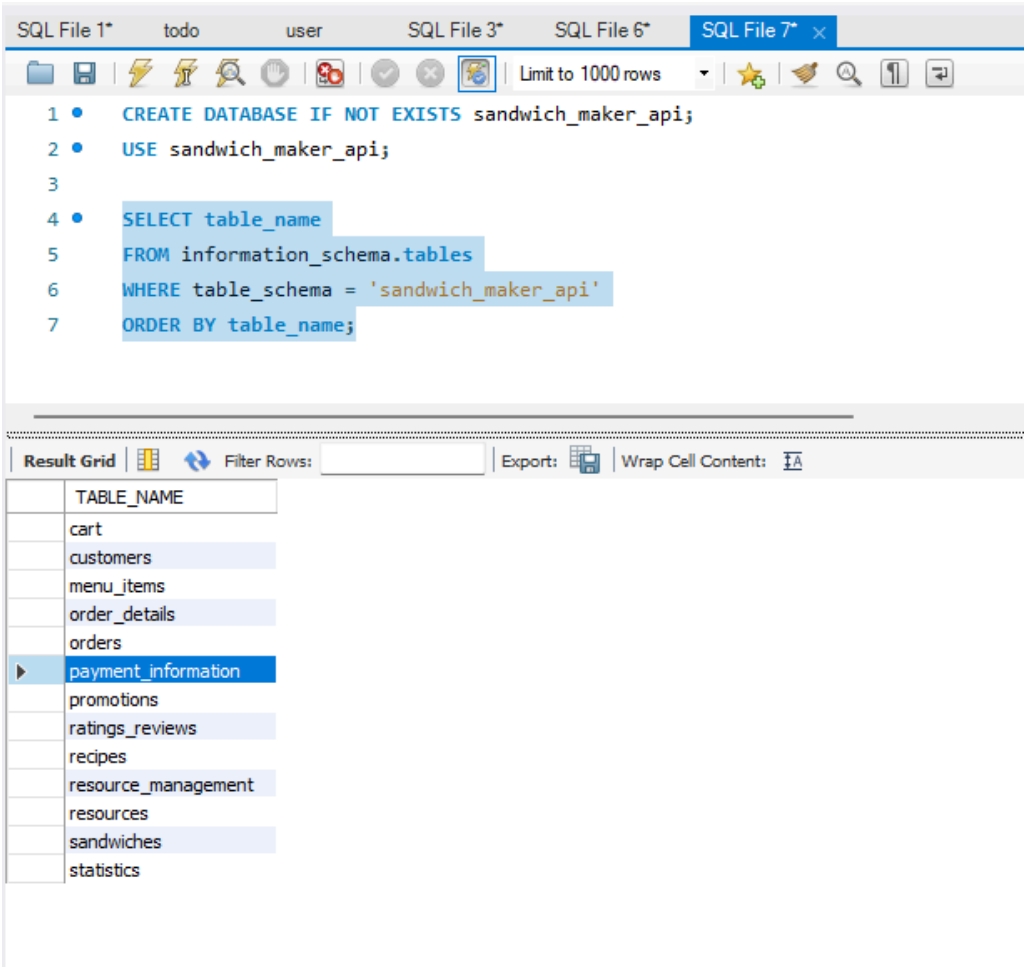



GET /resource-management/{item_id} Read One
Parameters
Cancel
Name
Description
item_id * required
integer
(path)
2
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/resource-management/2' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/resource-management/2
Server response
Code
Details
200
Response body
{
"amount_of_item": "25.00",
"unit": "string",
"id": 2
}
Download
Response headers
content-length: 49
content-type: application/json
date: Tue,28 Jul 2026 18:04:49 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}


PUT /resource-management/{item_id} Update
Parameters
Cancel
Reset
Name
Description
item_id * required
integer
(path)
2
Request body required
application/json
Edit Value | Schema
Execute
Clear
Responses
Curl
curl -X 'PUT' \
'http://127.0.0.1:8000/resource-management/2' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
"amount_of_item": 30.0,
"unit": "items"
Request URL
http://127.0.0.1:8000/resource-management/2
Server response
Code
Details
200
Response body
{
"amount_of_item": "30.00",
"unit": "items",
"id": 2
}
Download
Response headers
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
content-length: 48
content-type: application/json
server: uvicorn
vary: Origin
Responses
Code
Description
Links
200
Successful Response
No links
Media type

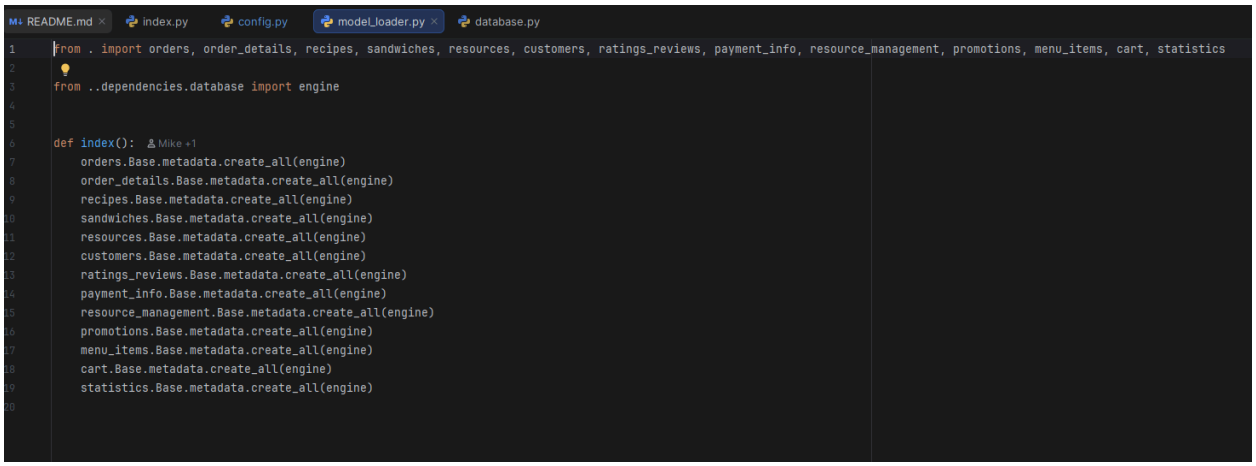

DELETE /resource-management/{item_id} Delete
Parameters
Cancel
Name
Description
item_id * required
integer
(path)
2
Execute
Clear
Responses
Curl
curl -X 'DELETE' \
'http://127.0.0.1:8000/resource-management/2' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/resource-management/2
Server response
Code
Details
204
Undocumented
Response headers
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
date: Tue,28 Jul 2026 18:07:20 GMT
server: uvicorn
vary: Origin
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}

Code Examples (Snippets):

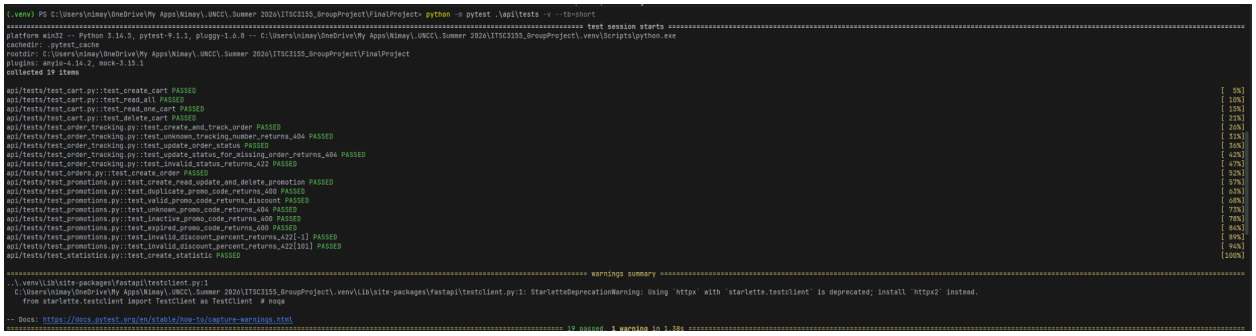


MySQL database verification showing all 13 tables created for the Sandwich Maker API.

(model_loader.api)



Full-Project PyUnit Testing:



Complete PyUnit/pytest execution showing that all project tests passed.

The complete project test suite was executed using pytest. Pytest automatically discovered and ran every test module inside the api/tests directory. The successful result confirms that all collected tests passed without failures or errors.

Development Environment:

In order to set up the development environment you must first choose your desired IDE. The most common/recommended IDE to use is PyCharm. Once in PyCharm create a new project and clone the GitHub repository. A link from the GitHub repository should suffice in terms of cloning this project.

The entire backend (models, schemas, controllers, and routers) are coded in Python. Python 3.10+ is required to run the project files.

Web frameworks include FastAPI and Uvicorn. FastAPI powers the routing, request validation, dependency injections, and auto-generates the SwaggerUI docs. Uvicorn is the server that runs the FastAPI application.

SwaggerUI is used to test every endpoint, inspect request/response schemas, and validate error handling. Using the SwaggerUI docs allows us to view and test each API endpoint. The link to use these docs are shown: http://127.0.0.1:8000/docs

The API design pattern follows a CRUD architecture, allowing each endpoint to have a Create, Read, Update, and Delete functionality. The layers of the project are as follows, models, schemas, controllers, and routers. The models are for SQLAlchemy definitions, Schemas are for pydantic request/response, controllers for logic and database operation, and routers for the FastAPI endpoint definitions.

The database used to store the tables is MySQL. The management and tool used for manual migrations and inspections is the MySQL Workbench. SQLAlchemy is required to define the models as Python classes and to manage the relationships between tables and queries. Within the project, there is a config file in which you must change the parameters for the SQL server to work on your own computer.

For version control, Git was used in order to have multiple developers working on the project. This allowed each individual section to be uploaded with little to no obstruction.

When initializing the project, run these commands to install required dependencies:
pip install fastapi
pip install “uvicorn[standard]”
pip install sqlalchemy
pip install pymysql
pip install pytest
pip install pytest-mock
pip install httpx
pip install cryptography

After running those commands in the terminal, you can proceed to run the server:
uvicorn api.main:app --reload